Formal Semantics and Conditional Exactness of State-Aware Transition Reuse

A proof for discrete transition systems with complete declared dependency closure

Ronnie Mack - Ronnie's Lab

July 2026

Abstract

This note gives a formal correctness result for State-Space Programming Methodology (SSPM). A deterministic discrete transition program is represented by a time-expanded dependency graph. A local intervention changes a set of source values. SSPM retains a baseline execution, computes the complete transitive closure of the intervention in the dependency graph, recomputes exactly that closure, and reuses baseline values outside it. Under sufficient state, deterministic transition semantics, complete dependency declaration, identical unaffected inputs and parameters, and complete output reconstruction, compact execution is equal to full from-scratch replay on every demanded output. The proof does not assume that the transition is globally linear. Linear-affine structure and ordered nonlinear residuals are admissible implementations only when their complete dependencies are represented.

1. Discrete transition programs

Let the declared system state at time t be

$$x_t \in \mathcal{X} = \prod_{i \in I} \mathcal{X}_i,$$

with exogenous input u_t, fixed parameters θ, and deterministic ordered transition

$$x_{t+1} = F_t(x_t, u_t; \theta), \qquad t = 0, \dots, T-1.$$

The state is **sufficient** when every value needed to determine future declared state and demanded outputs is contained in x_t, u_t, and θ. Hidden mutable state, undeclared I/O, nondeterministic reads, and unmodeled side effects violate this condition.

An implementation of F_t may contain intermediate values. We therefore model one rollout as an ordered collection of scalar or block assignments

$$z_v = \phi_v\left((z_p)_{p \in \mathrm{pa}(v)}\right),$$

where each node v identifies a state coordinate, input, parameter, or intermediate value at a particular logical time and program position. The parent set $\mathrm{pa}(v)$ contains every value read by the assignment. Ordering edges are included whenever program semantics require one assignment to precede another.

2. The time-expanded dependency graph

Definition 1: Declared dependency graph

The **time-expanded dependency graph** is the directed graph

$$G_T = (V_T, E_T),$$

where (p, v) belongs to E_T exactly when the assignment for v may depend on the value of p. Source nodes include the initial state, exogenous inputs, and parameters. Computed nodes include intermediate values, subsequent state, and demanded outputs.

For this proof, G_T is assumed to be acyclic after expanding the program's declared evaluation order. A transition with an iterative fixed-point solver must expose that solver as an ordered finite computation or supply a separate convergence argument.

Definition 2: Baseline execution

Given baseline sources s^0, evaluation in topological order produces baseline values

$$b_v = \mathrm{Eval}_v(s^0), \qquad v \in V_T.$$

The collection $B = (b_v)_{v \in V_T}$ is retained as the resident baseline.

Definition 3: Intervention and full replay

An intervention changes a set of source nodes Q. Let s^1 equal s^0 outside Q and contain the edited values on Q. Full from-scratch replay under s^1 produces

$$y_v = \mathrm{Eval}_v(s^1).$$

Definition 4: Complete affected closure

The affected closure is the set of computed nodes reachable from an edited source:

$$C = \mathrm{Reach}_{G_T}(Q) \cap V_T^{\text{computed}}.$$

The closure is **complete** when every semantic dependency is represented in E_T. If a changed source can influence a value through an undeclared read, alias, side effect, dynamic dispatch target, or omitted coupling edge, the closure is not complete.

3. Compact resident execution

Definition 5: Compact execution

Compact execution produces values c_v in topological order:

$$c_v = \begin{cases} s^1_v, & v \text{ is a source}, \\ \phi_v((c_p)_{p \in \mathrm{pa}(v)}), & v \in C, \\ b_v, & v \notin C \text{ and } v \text{ is computed}. \end{cases}$$

Thus nodes in the affected closure are recomputed and nodes outside it are reused from the baseline. A demanded result is reconstructed by projecting the mixture of recomputed and reused values.

4. Conditional exactness

Lemma 1: Unaffected-node invariance

For every computed node v outside C, full replay equals the baseline:

$$v \notin C \implies y_v = b_v.$$

Proof

Proceed in a topological order of G_T. Suppose v is outside C. No edited source in Q reaches v; otherwise v would belong to C. Consequently no parent of v can differ because of the intervention. If a parent p were reachable from Q, then the edge (p, v) would make v reachable as well. Source parents therefore have the same values under s^0 and s^1, while computed parents equal their baseline values by the induction hypothesis. Because ϕ_v is deterministic and receives identical arguments in both executions,

$$y_v = \phi_v((y_p)_p) = \phi_v((b_p)_p) = b_v.$$

This establishes the claim for every unaffected node. □

Lemma 2: Recomputed-node equality

For every node v in the affected closure, compact execution equals full replay:

$$v \in C \implies c_v = y_v.$$

Proof

Again proceed in topological order. Edited source nodes receive the same intervention values in compact execution and full replay. Let v be a computed node in C, and assume equality for all earlier affected nodes. For each parent p of v:

- if p is affected, $c_p = y_p$ by induction;
- if p is unaffected, $c_p = b_p = y_p$ by Lemma 1.

The complete parent tuple supplied to ϕ_v is therefore identical in compact execution and full replay. Determinism gives

$$c_v = \phi_v((c_p)_p) = \phi_v((y_p)_p) = y_v.$$

Hence equality holds for every recomputed node. □

Theorem 1: Conditional exactness of state-aware reuse

Let a finite-horizon discrete transition program satisfy:

1. the declared state is sufficient;
2. transition assignments are deterministic under fixed sources;
3. the time-expanded dependency graph is complete;
4. unaffected sources, parameters, and exogenous inputs are identical to the baseline;
5. affected assignments use the same ordered semantics as full replay; and
6. demanded outputs are reconstructed from all required affected and unaffected values.

Then compact resident execution equals full from-scratch replay on every demanded output d:

$$c_d = y_d.$$

Proof

Every demanded node is either in C or outside C. If it belongs to C, its compact value equals full replay by Lemma 2. If it is outside C, its reused baseline value equals full replay by Lemma 1. Complete output reconstruction selects the appropriate value in either case, so every demanded output agrees with full replay. □

Corollary 1: Full-state equality

If every state coordinate over the requested horizon is demanded, the complete compact trajectory equals the full replay trajectory:

$$X_{0:T}^{\text{compact}} = X_{0:T}^{\text{full}}.$$

Corollary 2: Contract and decision equality

Let a deterministic state contract be

$$K = \Phi((y_d)_{d \in D})$$

for demanded set D, and let a downstream decision be $a = \pi(K)$. Under the theorem's assumptions,

$$K^{\text{compact}} = K^{\text{full}}$$

and therefore

$$a^{\text{compact}} = a^{\text{full}}.$$

This result concerns semantic equality. Whether the contract is sufficient for the real decision objective is a separate modeling question.

5. Linear and nonlinear transition forms

The exactness theorem does not require linearity. A supported transition may be written as

$$x_{t+1} = A_t x_t + B_t u_t + b_t + H_t(x_t, u_t, L_t),$$

where

$$L_t = A_t x_t + B_t u_t + b_t$$

and H_t contains ordered nonlinear residual operations such as masks, thresholds, clipping, products, or intermediate-dependent updates. This decomposition is useful for implementation and acceleration only when the dependencies of both the affine base and the residual are declared.

Proposition 1: Nonlinearity does not invalidate closure exactness

If every operation in H_t is deterministic and its complete read/write and ordering dependencies are included in G_T, Theorem 1 applies unchanged.

Proof

The theorem is stated over arbitrary deterministic node functions ϕ_v, not only affine functions. Nonlinear residual operations are therefore ordinary nodes in the same dependency graph. Completeness and topological induction, not linearity, establish equality. □

Accordingly, SSPM does **not** claim that arbitrary systems are globally linearizable. Measurements such as "linear coverage" may describe a selected decomposition, but they do not prove semantic correctness or universal acceleration.

6. Cost boundary

Let $w(v)$ denote the cost of evaluating node v. Let closure discovery, resident lookup, and demanded-output reconstruction cost T_{closure}, T_{lookup}, and $T_{\text{materialize}}$. Then

$$T_{\text{compact}} = T_{\text{closure}} + T_{\text{lookup}} + \sum_{v \in C} w(v) + T_{\text{materialize}},$$

while full replay costs approximately

$$T_{\text{full}} = \sum_{v \in V_T^{\text{computed}}} w(v).$$

Proposition 2: Exactness does not imply speed

Compact execution is economically useful only when

$$T_{\text{compact}} < T_{\text{full}}.$$

Dense closure, cold compilation, expensive closure discovery, or full-output materialization can reverse the inequality even though Theorem 1 still holds. The full-replay path is therefore part of the runtime contract.

7. Floating-point interpretation

Theorem 1 gives exact equality when compact execution and full replay use the same deterministic operations, ordering, and arithmetic semantics. Different fusion, vectorization, or backend reduction orders can produce floating-point rounding differences. In that setting the implementation claim must be stated as equality within a declared numerical tolerance, supported by differential tests. Bitwise equality must not be inferred from mathematical equivalence.

8. Failure boundary

The proof does not apply when any premise fails. Important counterconditions include:

- hidden state or undeclared side effects;
- incomplete read, write, alias, graph-coupling, or ordering dependencies;
- changed exogenous inputs or parameters outside the intervention declaration;
- nondeterministic transition behavior;

- incompatible baseline and replay semantics;
- incomplete demanded-output reconstruction;
- unconverged or differently ordered iterative transitions; and
- unsupported mutation that is accepted instead of rejected.

When these conditions cannot be ruled out, SSPM must reject the compact path or execute full replay.

9. Distilled result

The formal contribution is:

$$\boxed{\text{complete declared closure} + \text{sufficient deterministic state} \Longrightarrow \text{full-replay-equivalent demanded outputs}}$$

The performance contribution is conditional:

$$\boxed{\text{reuse is useful only when closure and reconstruction cost less than replay}}$$

The resulting methodology is not universal linearization. It is exact local reuse for declared discrete transition programs, with rejection and full replay as correctness-preserving outcomes.